



Learn Claude — A Hands-On Notebook Course

8 Modules · 57 Runnable Jupyter Notebooks · Beginner-Friendly

A self-paced, code-first course for learning to build applications with Anthropic's Claude. Every lesson is a small Jupyter notebook with a clear purpose, fully documented functions, and runnable examples.

How the course is organized

The material is split into 8 modules. Modules 1–5 teach the core skills one topic per notebook; Module 6 is a set of feature deep-dives (with quick overviews of modules 1–5 up front) plus a medical-pipeline capstone; Module 7 is exam preparation for the Claude Certified Architect – Foundations (CCA-F) certification, one notebook per recommended practice area with its exam-domain weight; Module 8 is an applied case study that rebuilds the medical pipeline once per agentic architecture pattern, so you can compare them on the same problem. A shared **setup.ipynb** holds the one-time setup — every notebook opens with `%run ../setup.ipynb`, which loads the client, model id, and a simple `ask()` helper.

Module	Focus	Notebooks
Module 1	Agent Fundamentals	6
Module 2	Tools & MCP	5
Module 3	Claude Code Configuration	4
Module 4	Reliable Outputs	5
Module 5	Context & Escalation	5
Module 6	Agentic Systems Deep Dive	14
Module 7	Exam Preparation	9
Module 8	Applied Case Study — Medical	9

What every notebook contains

- **Purpose** — a short, plain-English statement of what you'll learn.
- **Fully documented code** — every function has a Purpose/Args/Returns docstring, and lines are commented.
- **Runnable examples** — small, simple demos of the real Claude feature (no advanced tricks).
- **Practical use** — where the idea is used in real projects.
- **Practice** — a short exercise with a built-in check so you know it works.

Module 1 — Agent Fundamentals

#	Notebook	You learn
1.1	Agentic Loops & Core API	Send messages; the agent loop driven by stop_reason
1.2	Multi-Agent Orchestration	Split a big job across helper agents
1.3	Hooks & Programmatic Enforcement	Enforce rules in code, not just prompts
1.4	Session Management & Workflows	End in a known state, hand off, resume
1.5	Use case — a refund agent that survives a logout	Capstone: refund agent with human approval that survives a logout
1.6	Parallel Tool Calls in One Turn	Independent tool calls in one turn — one round trip, not three

Module 2 — Tools & MCP

#	Notebook	You learn
2.1	Tool Description Best Practices	Descriptions decide which tool Claude picks
2.2	Structured Error Responses	Structured errors; 'broke' vs 'found nothing'
2.3	Tool Distribution & Selection	Few, single-purpose tools; tool_choice
2.4	MCP Server Configuration	MCP config, secrets, local vs remote
2.5	Built-in Tools	Grep / Glob / Read / Write / Edit / Bash

Module 3 — Claude Code Configuration

#	Notebook	You learn
3.1	CLAUDE.md Hierarchy & Configuration	CLAUDE.md layers; hooks for must-hold rules
3.2	Custom Commands & Skills	Slash commands vs auto-triggered skills
3.3	Plan Mode & Iterative Refinement	Plan mode vs direct; refine with examples
3.4	CI/CD Integration & Batch Processing	Headless JSON output; path-scoped rules

Module 4 — Reliable Outputs

#	Notebook	You learn
4.1	Explicit Criteria & Instruction Design	Say what to include, ignore, and the shape
4.2	Few-Shot Prompting	Few-shot examples for tricky cases
4.3	Tool Use for Structured Output	Tool + schema; nullable + enum with 'unclear'
4.4	Validation-Retry Loops & Multi-Pass Review	Valid shape != true; retry, fresh review
4.5	Fixing Truncated JSON with Split-and-Merge	Split-and-merge beats raising max_tokens at the output ceiling

Module 5 — Context & Escalation

#	Notebook	You learn
5.1	Context Optimization & Positioning	Edges beat the middle; keep exact numbers
5.2	Escalation & Error Propagation	Escalate on real signals, not tone
5.3	Context Degradation & Extended Sessions	Context rot; pin facts, compress, refresh
5.4	Human Review & Information Provenance	Review all high-stakes; add provenance
5.5	Server-Side Compaction (and Pausing to Keep Recent Turns)	Server-side compaction; pause to keep recent turns intact

Module 6 — Agentic Systems Deep Dive

#	Notebook	You learn
01	Agentic Systems — Overview	Quick recap of Module 1
02	Tools & MCP — Overview	Quick recap of Module 2
03	Claude Code Configuration — Overview	Quick recap of Module 3
04	Structured Output — Overview	Quick recap of Module 4
05	Context & Reliability — Overview	Quick recap of Module 5
06	Claude Managed Agents	Agents run by Anthropic (agents/sessions/events)
07	Managed Agents — Multi-Agent	Orchestrator + worker agents
08	Managed Agents — A Real-World Example	A real 4-step code-review flow
09	A Multi-Stage Pipeline (Medical Notes Example)	Chain small Claude steps into a pipeline
10	Assistant Prefill	Seed the answer to force clean output
11	RAG End-to-End	Retrieve, augment, generate over your docs
12	Safety & Responsible Deployment — Overview	Guardrails at the tool boundary; 'code must, prompt should'
13	Anthropic Claude — Medical Multi-Agent Pipeline	Capstone: a sequential medical multi-agent pipeline, pure Anthropic SDK
14	Claude Managed Agents & Multi-Agent Orchestration	Managed agents and multi-agent orchestration, end to end

Module 6 also includes capstone material: **CAPSTONE_medical_pipeline_plan.md** — the written build plan behind the medical pipeline.

Module 7 — Exam Preparation

#	Notebook	You learn
7.1	Build an Agent with the Claude Agent SDK	Agent loop with the Claude Agent SDK (Domain 1, 27%)
7.2	Configure Claude Code for a Real Project	CLAUDE.md hierarchy, path rules, skills, MCP (Domain 3, 20%)
7.3	Design and Test MCP Tools	Differentiating descriptions; loud structured failures (Domain 2, 18%)
7.4	Build a Data Extraction Pipeline	tool_use + JSON schema; validate/retry; batch (Domain 4, 20%)
7.5	Practice Prompt Engineering	Few-shot, explicit criteria, multi-pass review (Domain 4, 20%)
7.6	Study Context Management Patterns	Extract facts, scratchpad files, delegate to subagents (Domain 5, 15%)
7.7	Understand Escalation & Human-in-the-Loop	When to stop and ask a human; route by confidence (Domain 5, 15%)
7.8	Build Agents, Coordinators & Subagents	Specialists, coordinator, orchestrator/worker patterns
7.10	Automated Test Generation Quality	Tests that pass but assert nothing — judging generated-test quality

Module 8 — Applied Case Study — Medical

#	Notebook	You learn
00	Agentic Architecture Patterns — a medical review pipeline	The pattern map: which architecture the medical pipeline needs, and why
01	Prompt chaining — the baseline	Workflow, no autonomy — fixed steps, each feeding the next
02	Parallelization (sectioning)	Sectioning — run independent stages at once and fan results back in
03	Evaluator–optimizer (generator / critic loop)	A generate → critique → revise loop with a quality gate
04	Routing — a classifier front door	Classify first, then send the case down the right specialist path
05	Tool use — the real agent loop	Give the model tools and let the loop decide the steps
06	Orchestrator–workers	A coordinator that plans the work and delegates it
07	Multi-perspective ensemble (voting)	Several independent reviewers, then reconcile the disagreement
08	Putting it together — the target architecture	Assembling the patterns into the production design

The case study ships **common.py** (shared client and helpers) and a **README.md** walking through the pattern comparison.

At a glance: 8 modules · 57 runnable notebooks · one shared setup · every function documented · one small practice check per lesson · exam-prep module mapped to CCA-F domains. Built for students learning to build with Claude from zero.

An AmbariCloud Data & AI learning resource.